

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Project ID: 25-26J-477

Project Proposal Report

AnneSiyani Srikanthan – IT22125248

BSc (Hons) in Information Technology Specialising in Software
Engineering

Department of Computer Science and Software Engineering,
Faculty of Computing, Sri Lanka Institute of Information
Technology (SLIIT)

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

Project ID: 25-26J-477

Project Proposal Report

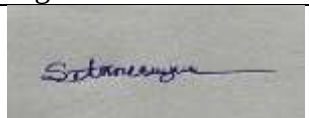
BSc (Hons) in Information Technology Specialising in Software
Engineering

Department of Computer Science and Software Engineering,
Faculty of Computing, Sri Lanka Institute of Information
Technology (SLIIT)

August 2025

Declaration

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the next.

Student ID	Name	Date	Signature
IT22125248	Annesiyani Srikanthan	15/08/2025	

Above candidate is carrying out research for the undergraduate dissertation under our supervision.

	Title	Name	Date	Signature
Supervisor	Mr.	Vishu Jayasinghe	14/08/2025	
Co-Supervisor	Ms.	Kaushalya Rajapakse	16/08/2025	

Acknowledgement

I would like to express my deep gratitude for all the people who helped me to continue my 4th year research project. First, I greatly appreciate our Research Project supervisor Ms. Vishan Jayasinghearachchi, who provided the guidance and encouragement to continue this project successfully. I am also grateful to our co-supervisor Ms. Kaushalya Rajapaksa for the valuable and constructive suggestions given during the planning of our research work.

I am also thankful to my project teammates for their collaboration and support, which created a productive and motivating research environment. Special appreciation goes to the Department of **Software Engineering, SLIIT**, for providing the resources and facilities necessary to carry out this work.

Finally, I would like to extend my heartfelt appreciation to my family and friends for their constant encouragement and moral support during this research journey.

Abstract

The purpose of this study is to develop a human-like, screen-aware task automation system capable of performing tasks on software interfaces without requiring internal system access, APIs, or installations. Traditional automation tools rely heavily on view hierarchies, accessibility APIs, or fixed scripts, which limits their applicability to black-box testing, cross-platform environments, and legacy systems. This research addresses the gap by proposing an external, adaptive agent that observes the screen visually, interprets natural language instructions, and executes actions via HID emulation, thereby mimicking human interaction with the system.

The study employs an integrated design where visual perception, language-based reasoning, and action execution are coordinated in a continuous feedback loop. The agent uses large language or vision-language models to convert instructions into step-by-step actions, adapts dynamically to unexpected outcomes through retries and exploratory testing, and maintains detailed logs and structured reports. Integration with tools like Jira ensures traceable recording of test outcomes, errors, and progress.

Expected outcomes include an autonomous system capable of performing reliable black-box testing with human-like adaptability, handling failures intelligently, and producing structured feedback for evaluation. This approach overcomes the limitations of conventional automation methods by providing a flexible, feedback-driven, and traceable testing process that operates entirely externally, supporting a wide range of platforms and applications. The study demonstrates that intelligent agentic automation can enhance testing efficiency, reduce manual intervention, and enable exploratory testing in scenarios where traditional automation tools fail.

Table of Contents

1	Introduction.....	6
2	Background & Literature Survey	8
2.1	Background	8
2.2	Literature Survey	8
3	Research Gap	10
4	Research Problem	11
5	Objectives	12
5.1	Main Objectives.....	12
5.2	Specific Objectives.....	12
6	Methodology	13
7	Project Requirements	17
7.1	Functional Requirements.....	17
7.2	Non-Functional Requirements.....	17
7.3	Expected Test Cases	18
8	Budget and Budget Justification.....	19
9	References.....	21
10	Appendices.....	23

1 Introduction

Overall Background of the Research

Task automation has become integral to modern software development and testing, aiming to enhance efficiency and reduce human error. Traditional automation tools, such as Selenium [1] and Appium [2], rely on predefined scripts and internal system access, which limits their adaptability and applicability in black-box testing scenarios. Recent advancements have introduced AI-driven approaches, enabling more dynamic and intelligent automation processes. For instance, Apple researchers have developed AI systems that understand screen context, facilitating more natural interactions with voice assistants [3].

Relevant Studies and Techniques

To undertake this project, proficiency in several areas is essential:

- **Computer Vision:** Techniques like Optical Character Recognition (OCR) and image processing are crucial for screen perception.
- **Natural Language Processing (NLP):** Understanding and generating human language to interpret instructions.
- **Human-Computer Interaction (HCI):** Designing interfaces that allow seamless interaction between humans and machines.
- **Machine Learning:** Implementing algorithms that enable the system to learn from data and improve over time.
- **Software Testing Methodologies:** Understanding various testing strategies, including black-box testing, to evaluate system performance effectively.

Current State of the Art

The integration of AI into task automation is progressing rapidly. Systems like Manus AI combine computer vision and NLP to perform tasks across any user interface by observing the screen [4]. Similarly, VisionTasker employs a two-stage framework combining vision-based UI understanding and LLM task planning for mobile task automation [5]. These advancements indicate a shift towards more adaptive and intelligent automation systems.

Existing Solutions and Research Gaps

Traditional tools like Selenium and Appium automate testing using predefined scripts and system access [1],[2]. They are effective for repetitive tasks but cannot handle unexpected UI changes or black-box scenarios. Recent AI-driven systems, such as Manus AI [4] and VisionTasker [5], combine computer vision and language models to perform tasks without internal APIs. While more adaptable, they still struggle with dynamic errors, cross-platform generalization, and tool integration.

Research Gaps:

- **Adaptability:** Systems fail when unexpected changes occur.
- **Generalization:** Many require retraining for new applications.
- **Integration & Traceability:** Few provide automatic logging or Jira integration.
- **Black-Box Testing:** Existing tools often need internal access rather than external observation.

Our Approach:

This research proposes a human-like, screen-aware agent combining: visual perception, LLM/VLM reasoning, HID actions, and continuous feedback loops. It performs adaptive, exploratory testing, handles failures, and logs results in Jira, overcoming limitations of current tools and enabling robust black-box testing.

2 Background & Literature Survey

2.1 Background

Software testing is a critical component of the software development lifecycle, ensuring that applications function correctly and reliably. Traditional automation tools, such as Selenium[1] and Appium[2], rely on predefined scripts and internal system access to perform repetitive tasks. While effective in controlled environments, these tools have limited adaptability, struggle with black-box testing, and cannot handle legacy or cross-platform applications.

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), and computer vision have introduced intelligent automation systems capable of observing screens, interpreting natural language instructions, and making dynamic decisions similar to human testers. Systems like Manus AI [4] and VisionTasker [5] demonstrate the potential of AI-driven task automation, but challenges remain in generalization, dynamic error handling, and seamless integration with reporting tools.

There is a clear need for a fully external, human-like automation agent that can perceive the screen visually, plan and execute tasks adaptively, handle unexpected failures, and maintain detailed logs for traceability. This research proposes developing an Agentic AI-based system that combines visual perception, AI reasoning, HID-based action execution, and continuous feedback loops, addressing current limitations and enabling robust, adaptive black-box testing across platforms.

2.2 Literature Survey

This research integrates multiple advanced techniques to build a human-like, screen-aware automation agent. The Literature Survey is organized by individual components to highlight the current state-of-the-art and research gaps.

2.2.1 Agentic AI Logic

The core of adaptive task automation relies on intelligent decision-making. Studies in autonomous agents and adaptive exploratory testing show that systems capable of perceiving outcomes and adjusting plans dynamically achieve higher success in unknown or changing environments [6]. Traditional rule-based systems or fixed test

scripts cannot handle unexpected changes or task failures effectively. Agentic AI, as proposed in this research, builds on these concepts by implementing a continuous feedback loop (Perceive -> Plan -> Act -> Evaluate), mimicking human-like reasoning and improving resilience to failures.

Research Gap: Existing solutions rarely integrate adaptive decision-making with full external perception and reporting mechanisms like Jira.

2.2.2 Visual Perception (Web Cam-Based)

Computer vision techniques have been widely applied to UI recognition and screen understanding. OCR tools like Tesseract and EasyOCR extract textual information from interfaces [7], while OpenCV enables image preprocessing and element detection. Existing AI-based UI automation tools such as VisionTasker [5] demonstrate the use of vision for mobile task automation.

Research Gap: Most approaches require internal APIs or system access. A fully external, camera-driven perception module enabling real-time feedback and structured data for intelligent decision-making is still limited.

2.2.3 HID Actuation (ESP32-Based)

Hardware-driven input simulation allows systems to interact with devices without relying on OS-level software hooks. ESP32-based HID emulation has been explored for keyboard/mouse automation in embedded systems [8]. However, integrating HID actuation with adaptive AI agents for real-time, synchronized testing is largely unexplored.

Research Gap: Current tools rarely provide bidirectional communication between the actuation layer and AI logic for feedback-driven retries and error handling.

2.2.4 LLM & Prompt Engineering Module

Recent work in LLM-driven task automation shows that large language models can convert natural language instructions into step-by-step executable actions [9]. Combining LLM reasoning with visual perception enables context-aware UI interaction. Prompt engineering techniques improve accuracy and adaptability by designing model inputs to reflect desired actions.

Research Gap: Most studies focus on text-based or internal API-driven automation. External, vision-guided LLM-based automation remains underexplored, especially in black-box scenarios where system internals are inaccessible.

Summary of Gaps and Novelty

- Across all components, the literature reveals:
- Limited external, camera-based perception for UI automation.
- Lack of human-like adaptive decision-making in failure scenarios.
- Minimal integration of AI reasoning, actuation, and reporting tools in a unified framework.

Novelty of This Research: The proposed system combines Agentic AI, vision-based perception, LLM-driven planning, and HID actuation into a single, adaptive, and traceable automation agent, addressing the limitations of current solutions and enabling robust black-box testing.

3 Research Gap

Despite recent advancements in AI-driven automation and screen-aware task execution, several gaps remain:

1. Adaptability and Human-Like Decision Making
 - Traditional automation tools rely on fixed scripts and fail to handle unexpected changes or errors[1][2].
 - Existing AI-based agents often lack dynamic exploratory testing and real-time adaptive decision-making.
2. External Perception and Black-Box Testing
 - Most automation solutions depend on internal APIs, view hierarchies, or system access [1][2].
 - Fully external, camera-based perception for black-box testing remains underdeveloped.
3. Integration of Modules
 - Few systems integrate perception, reasoning, and actuation in a single cohesive framework [10].
 - Automated feedback loops and bidirectional communication between AI logic, HID actions, and visual input are rarely implemented.

4. Traceability and Reporting

- Existing approaches often lack seamless integration with issue-tracking or reporting tools such as Jira [10].
- Automated logging of test outcomes, errors, and progress for traceable testing is limited.

Conclusion:

Current research partially addresses individual components of intelligent automation (vision, LLM reasoning, HID actuation) but does not provide a fully integrated, adaptive, human-like agent for robust black-box testing. This research aims to bridge these gaps by combining visual perception, Agentic AI logic, LLM-driven planning, and HID-based interaction with continuous feedback and Jira integration, creating a comprehensive and adaptive automation system.

4 Research Problem

Software test automation has traditionally relied on system-level access (e.g., APIs, accessibility services, or internal view hierarchies) to execute tasks and validate results. While effective in controlled environments, these methods face major limitations in black-box testing and cross-platform scenarios where internal access is restricted or unavailable.

Existing AI-driven test automation tools (e.g., TestRigor [10]) show progress by incorporating machine learning for test generation and reporting. However, they still suffer from critical shortcomings:

1. **Limited Adaptability** – Many solutions fail when encountering unexpected UI changes, dynamic screen layouts, or execution errors, as they lack human-like reasoning and adaptive retries.
2. **Weak Integration Across Modules** – Few systems integrate visual perception, reasoning (AI logic), and HID-based actuation in a single cohesive framework with real-time feedback loops.
3. **Poor Traceability** – Automated and seamless integration with issue tracking tools like Jira is limited; most require manual intervention for error logging or reporting.
4. **Generalization Issues** – Current tools often need extensive retraining or customization to work across different applications, platforms, and environments.

As a result, there is no existing solution that provides a fully autonomous, human-like, screen-aware AI agent capable of:

- Observing and interpreting the screen externally,
- Reasoning and adapting actions dynamically,
- Executing tasks via HID emulation, and
- Automatically logging results in traceable issue-tracking systems.

This gap necessitates the development of a novel agentic AI-based framework that unifies perception, reasoning, and action within a robust feedback-driven loop, enabling adaptive, traceable, and platform-independent test automation.

5 Objectives

5.1 Main Objectives

The main objective of this research is to design and develop a human-like, screen-aware Agentic AI framework for software testing that operates entirely through external observation and interaction. By integrating visual perception via a camera, intelligent reasoning through LLM-based task planning, HID-based actuation for executing inputs, and automated traceability through seamless reporting, the system aims to enable adaptive, platform-independent black-box testing. Unlike traditional tools that depend on system-level access, APIs, or predefined scripts, this approach focuses on real-time perception, reasoning, and feedback-driven execution, allowing the agent to dynamically handle unexpected outcomes, mimic human testers, and provide end-to-end traceable reports for improved reliability and efficiency in software quality assurance.

5.2 Specific Objectives

1. **Agentic AI Logic:**
To develop the core decision-making engine that drives the system's intelligent behavior by interpreting natural language and visual instructions, managing the Perceive → Plan → Act → Evaluate control loop, and supporting adaptive exploratory testing. This module will handle retries,

maintain test states, coordinate with other components, and integrate with Jira for automated logging, reporting, and traceability.

2. Visual Perception:

To implement an external camera-based perception system using computer vision and OCR techniques to capture and analyze the screen in real time. This module will detect UI elements, track state changes, and provide structured visual feedback to enable intelligent reasoning, supporting black-box automation without system-level access.

3. HID Actuation:

To design and implement an ESP32-based Human Interface Device (HID) emulator that simulates human-like keyboard and mouse actions. This component will receive structured commands from the Agentic AI, execute them on the target system, and provide feedback on execution status, ensuring reliable and synchronized interaction with visual perception.

4. LLM & Prompt Engineering Module:

To integrate LLMs (or VLMs) with prompt engineering techniques for translating high-level natural language instructions into step-by-step executable actions. By combining reasoning with visual perception, this module will generate structured action plans that adapt dynamically to screen changes, enabling flexible, human-like test automation.

6 Methodology

This research adopts a modular and iterative methodology to design, implement, and evaluate a human-like, screen-aware Agentic AI framework for black-box task automation. The methodology is organized into system design, module development, integration, testing, and evaluation, supported by proper resource planning, task allocation, and timelines.

6.1 System Diagram [Appendix A]

The proposed system consists of four interconnected modules:

1. Visual Perception – Captures the target screen using an external camera, processes frames using OpenCV and OCR, and provides structured UI data.
2. Agentic AI Logic – Acts as the intelligent controller, executing the Perceive → Plan → Act → Evaluate loop.
3. LLM & Prompt Engineering – Converts natural language test instructions into structured action steps, integrated with perception data for accurate execution.
4. HID Actuation – Uses ESP32-S3-based HID emulation to perform keyboard/mouse actions on the system under test.

The modules communicate in a continuous feedback loop:

- Visual Perception → Agentic AI → LLM → HID → Visual Perception (verification) → Logging & Reporting (Jira).

6.2 Research Process (How the Project Will Be Carried Out)

The project will be carried out in five main phases:

1. Requirement Analysis & Literature Review – Identify gaps in existing automation systems and finalize tools/technologies.
2. Module Development – Develop perception, reasoning, HID, and LLM components individually.
3. System Integration – Connect all modules under Agentic AI Logic with feedback loop control.
4. Testing & Evaluation – Conduct experiments, benchmark against Selenium/Appium, and test robustness under unexpected UI changes.
5. Documentation & Reporting – Maintain project logs, reports, and final research outcomes.

6.3 Tasks and Sub-Tasks

Main Task	Sub-Tasks	Responsible Member
Agentic AI Logic	Control loop design, error handling, adaptive re-planning, Jira integration	Siyani

HID Actuation	ESP32 setup, USB HID coding, command execution, synchronization feedback	Amith
LLM & Prompt Engineering	Prompt design, instruction translation, LLM/VLM integration	Tharushi
System Integration	Connect modules, design communication protocols, feedback testing	All
Evaluation & Reporting	Benchmarking, case studies, test reports, performance metrics	All (Annesiyani leads)

6.4 Resources and Materials Needed

Hardware:

- ESP32-S3 development board (for HID emulation).
- USB cables and testing machine (PC/laptop).
- External webcam.

Software & Libraries:

- OpenCV, Tesseract OCR (for perception).
- LLM APIs (OpenAI GPT, LLaMA, or VLMs if supported).
- Python for AI logic and integration.
- Arduino IDE / ESP-IDF for ESP32 programming.
- Jira API for reporting.

Computer Support:

- GPU-enabled machine for model inference.
- Cloud support (if needed) for large-scale testing.

6.5 Data Collection

The project does not require surveys or interviews. Instead, it relies on screen-based data collected through:

- Captured frames from real software applications (login forms, spreadsheets, web apps).
- Generated OCR data from screen content.

- Logs of user instructions and system outputs.

This data will be used to train/test the robustness of the perception and reasoning modules.

6.6 Timeline (Work Plan)

Phase	Tasks	Duration
August - October	Requirement analysis, literature survey, finalize tools	2 months
October - December	Module development (Perception, HID, LLM, AI logic separately)	2 months
January - February	System integration & internal testing	1 month
March	Evaluation against existing tools, case study testing	1 month
April	Refinement, error handling, report generation automation	1 month
May	Final documentation, thesis writing, presentation prep	1 month

6.7 Anticipated Results

The anticipated outcome of this research is a fully functional Agentic AI framework capable of:

- Automating GUI tasks without internal access (true black-box testing).
- Handling unexpected screen changes adaptively through perception-reasoning loops.
- Performing cross-platform testing using external devices.
- Automatically generating traceable logs and Jira reports.

In real-world application, this system can be used for software QA testing, regression testing, and exploratory testing, especially for legacy systems or applications where APIs and accessibility features are unavailable.

7 Project Requirements

7.1 Functional Requirements

The system must fulfill the following functional requirements:

1. Visual Perception
 - Capture screen frames using an external webcam or mobile device.
 - Detect UI components (buttons, text fields, labels) using OCR and image processing.
 - Provide structured data (element type, position, label).
2. Instruction Interpretation
 - Accept natural language instructions from users (e.g., “Login with username and password”).
 - Translate instructions into step-by-step action plans using LLM/prompt engineering.
3. Agentic AI Logic
 - Implement the Perceive → Plan → Act → Evaluate loop.
 - Manage task execution with retry logic and error handling.
 - Adapt dynamically when the expected UI state changes.
4. HID Actuation
 - Simulate mouse and keyboard actions using ESP32 HID emulator.
 - Execute actions on the target system in real time.
 - Provide execution confirmation back to the AI logic.
5. Feedback & Reporting
 - Continuously validate success/failure through updated visual perception.
 - Log execution history, screenshots, and error cases.
 - Integrate with Jira (or similar) for issue reporting and traceability

7.2 Non-Functional Requirements

The system should also meet the following quality attributes:

1. Performance
 - Response time between instruction and action execution ≤ 2 seconds.
 - Support real-time feedback loop within acceptable latency.

2. Reliability
 - Ability to recover from failures by retrying or replanning tasks.
 - Maintain synchronization between perception, AI logic, and HID.
3. Usability
 - Allow testers to provide instructions in plain natural language.
 - Provide clear, human-readable logs and reports.
4. Scalability
 - Support different applications (web, desktop, legacy systems).
 - Allow extension to multiple input devices or advanced vision models.
5. Portability
 - Operate independently of the target system (no software installation).
 - Work across platforms (Windows, Linux, macOS).
6. Security & Safety
 - Ensure HID emulation cannot execute malicious unintended commands.
 - Maintain data privacy by limiting capture to screen content only.

7.3 Expected Test Cases

To validate the system, the following test scenarios will be executed:

1. Login Automation
 - Instruction: “Open application, enter username, enter password, and click Login.”
 - Expected: Correct input in fields and successful login.
2. Form Filling
 - Instruction: “Fill the form with Name =‘John’, Age=‘25’, and click Submit.”
 - Expected: Correct data entered, form submitted successfully.
3. Error Handling
 - Instruction: “Click the Submit button.” but the button is hidden/renamed.
 - Expected: Agent retries, rechecks screen, and logs error with screenshot.
4. Cross-Platform Execution
 - Run the same test case on Windows and Linux systems.
 - Expected: Successful execution without platform dependency.
5. Jira Integration

- Instruction: “Run test and log result to Jira.”
 - Expected: Automated issue created in Jira with logs and screenshots.
6. Adaptive Testing
- When UI layout changes slightly (button shifts position, color changes).
 - Expected: System still detects the element and executes action.

8 Budget and Budget Justification

The budget is designed to cover essential hardware, software, and project-related resources required to successfully develop and evaluate the proposed system.

8.1 Estimated Budget

Item	Quantity	Unit Cost (LKR)	Total Cost (LKR)	Justification
ESP32 Development Board (with HID support)	2 units	2,000	4,000	Required to emulate HID (keyboard & mouse) for system actuation. A backup unit ensures continuity in case of hardware failure.
Webcam (HD quality)	1 unit	6,000	6,000	Used for external visual screen capture to support perception module.
Cables, Connectors, and Mounts	-	3,000	3,000	Needed for hardware connectivity, stability, and mounting of camera.
Cloud GPU/Compute Credits (Google Colab / AWS / Azure)	-	20,000	20,000	For training/fine-tuning AI models, running experiments, and large-scale testing.
Software Licenses (Optional)	-	5,000	5,000	Tools such as OCR libraries, Jira integration plugins, or visualization software.
Documentation & Printing	-	5,000	5,000	For proposal, final report, and presentation material printing.
Miscellaneous	-	5,000	5,000	Covers unexpected expenses.

Total Estimated Budget: ~ Rs. 48,000

8.2 Budget Justification

1. Hardware (ESP32, Webcam, Accessories)
 - Essential for building the HID actuation and visual perception modules.
 - ESP32 enables low-cost HID emulation for automation tasks.
 - Webcam/Smartphone provides reliable screen capture for AI-based perception.
2. Cloud/Compute Resources
 - Since training and testing AI models require high processing power, cloud credits will be used for experiments, model fine-tuning, and large dataset handling.
3. Software & Licenses
 - Covers OCR engines, Jira integration plugins, and specialized libraries not available in open-source.
4. Documentation & Printing
 - Necessary for producing project deliverables, submission reports, and final presentation material.
5. Miscellaneous
 - Reserved for unforeseen costs (extra cables, replacement parts, or additional storage).

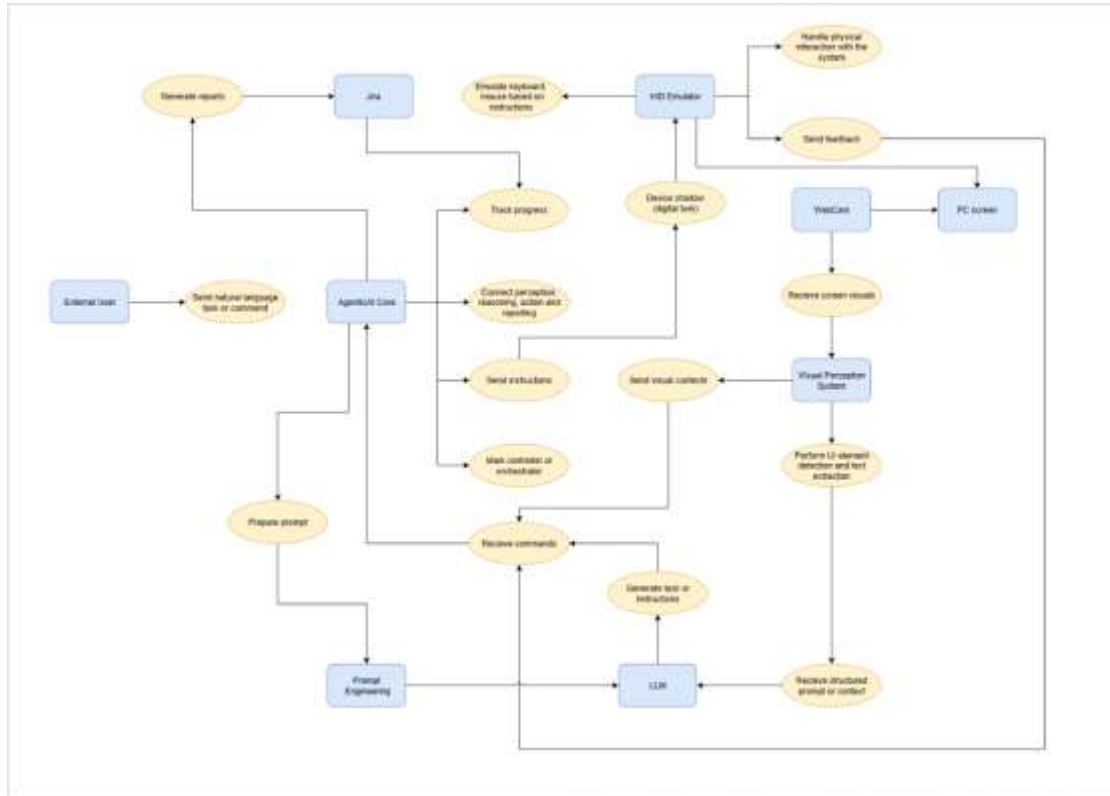
9 References

- [1] M. Kieliszek, I. Bano, and H. Zare, "A Comprehensive Review on Selenium and Its Effects on Human Health and Distribution in Middle Eastern Countries," *Biological Trace Element Research*, vol. 200, no. 3, pp. 971–987, Mar. 2022, doi: 10.1007/s12011-021-02716-z.
- [2] J. Wang and J. Wu, "Research on Mobile Application Automation Testing Technology Based on Appium," in *2019 International Conference on Virtual Reality and Intelligent Systems (ICVRIS)*, IEEE, Sep. 2019, pp. 247–250. doi: 10.1109/ICVRIS.2019.00068.
- [3] J. R. A. Moniz *et al.*, "ReALM: Reference Resolution As Language Modeling," Aug. 2024, [Online]. Available: <http://arxiv.org/abs/2403.20329>
- [4] M. Shen, Y. Li, L. Chen, and Q. Yang, "From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent," Jul. 2025, [Online]. Available: <http://arxiv.org/abs/2505.02024>
- [5] Y. Song, Y. Bian, Y. Tang, G. Ma, and Z. Cai, "VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, New York, NY, USA: ACM, Oct. 2024, pp. 1–17. doi: 10.1145/3654777.3676386.
- [6] N. Sun and J. Botev, "Intelligent autonomous agents and trust in virtual reality," *Computers in Human Behavior Reports*, vol. 4, p. 100146, Aug. 2021, doi: 10.1016/j.chbr.2021.100146.
- [7] D. R. Vedhaviyassh, R. Sudhan, G. Saranya, M. Safa, and D. Arun, "Comparative Analysis of EasyOCR and TesseractOCR for Automatic License Plate Recognition using Deep Learning Algorithm," in *2022 6th International Conference on Electronics, Communication and Aerospace Technology*, IEEE, Dec. 2022, pp. 966–971. doi: 10.1109/ICECA55336.2022.10009215.

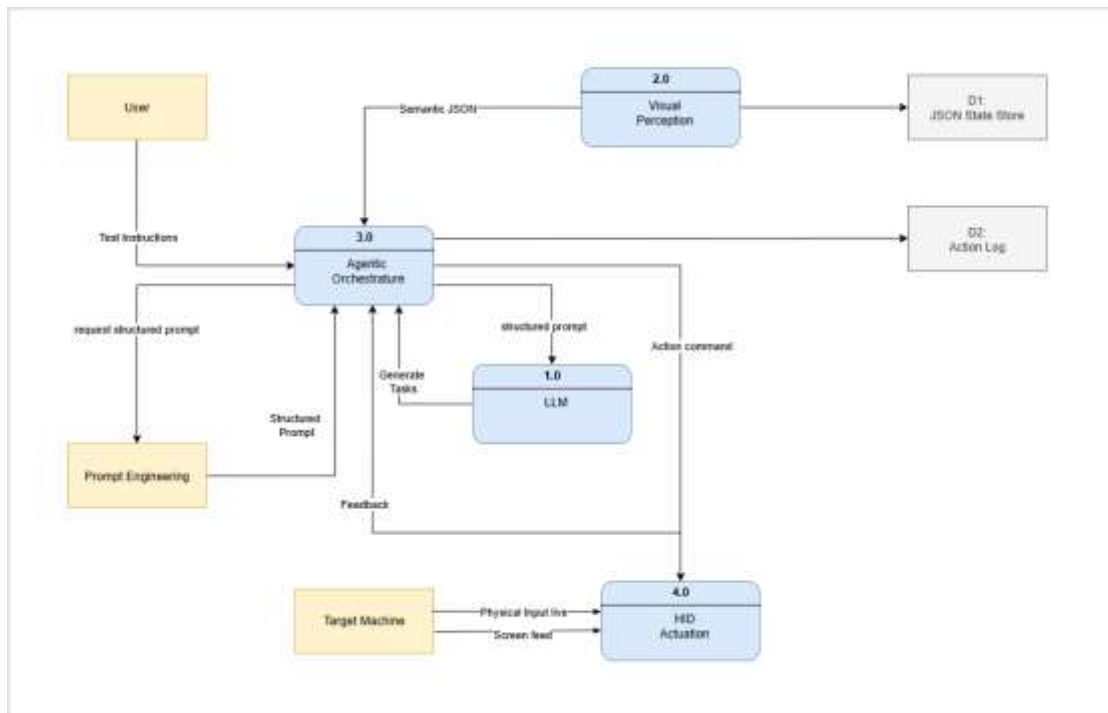
- [8] Muhammad Ryan Prayogi and Tommy, "Design and Development of Air Mouse Using ESP32 and MPU6050 Sensor," *Journal of Artificial Intelligence and Engineering Applications (JAIEA)*, vol. 4, no. 3, pp. 1678–1686, Jun. 2025, doi: 10.59934/jaiea.v4i3.983.\
- [9] L. Shen, Q. Yang, Y. Zheng, and M. Li, "AutoIoT: LLM-Driven Automated Natural Language Programming for AIoT Applications," Mar. 2025, [Online]. Available: <http://arxiv.org/abs/2503.05346>
- [10] testrigor, "testrigor case-studies," <https://testrigor.com/case-studies/>.

10 Appendices

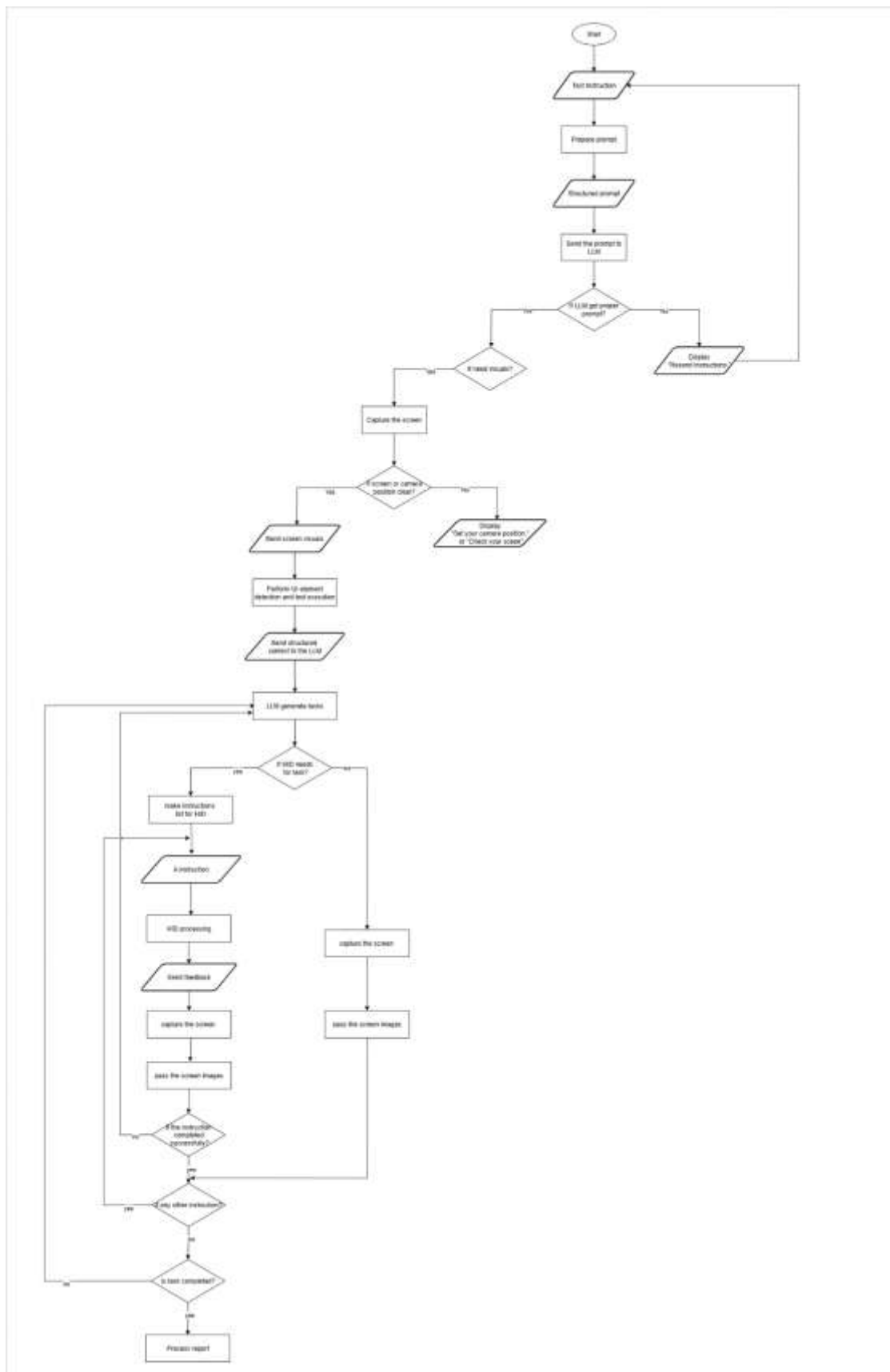
10.1 Appendix A: System Workflow Diagram



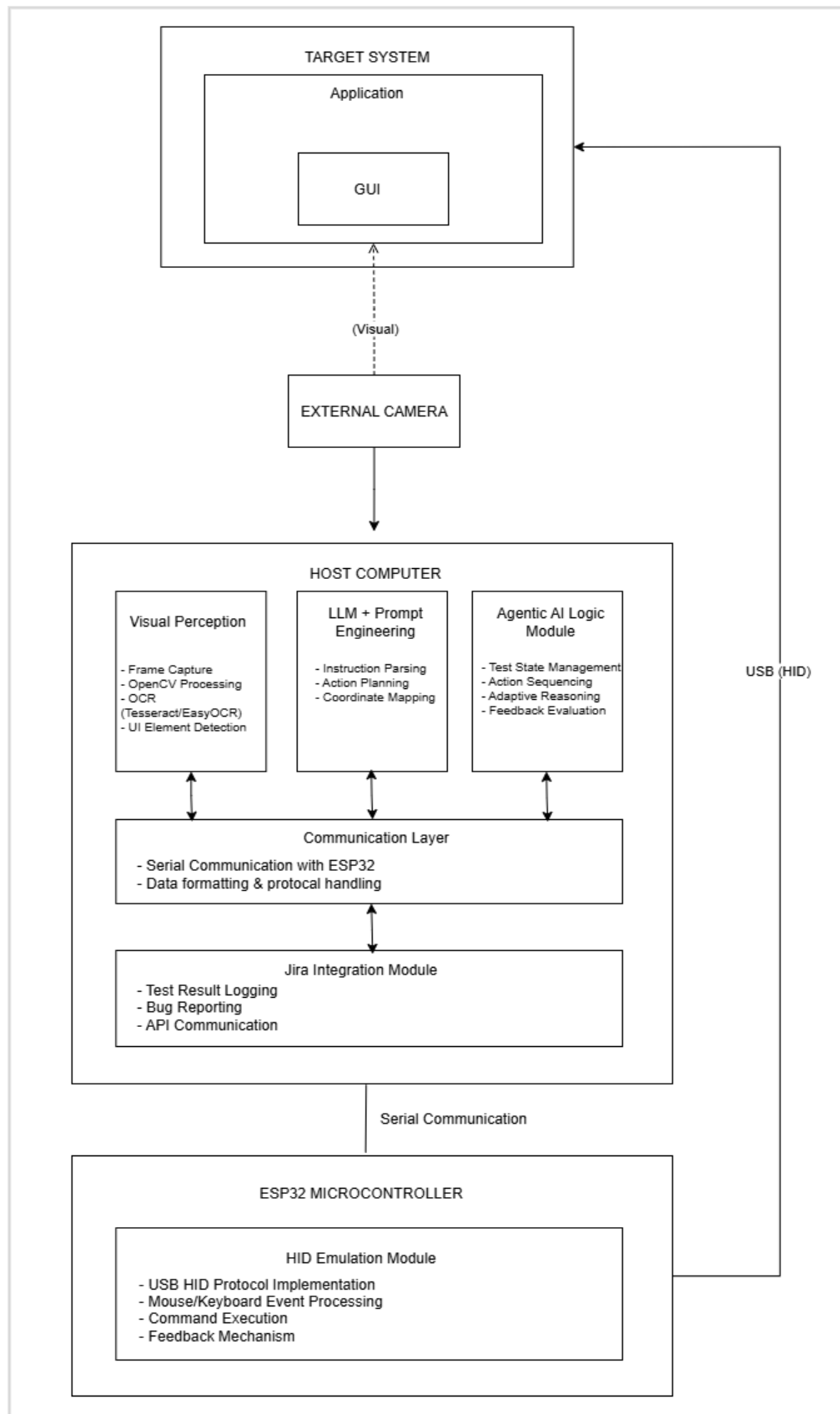
10.2 Appendix B: Level Diagram



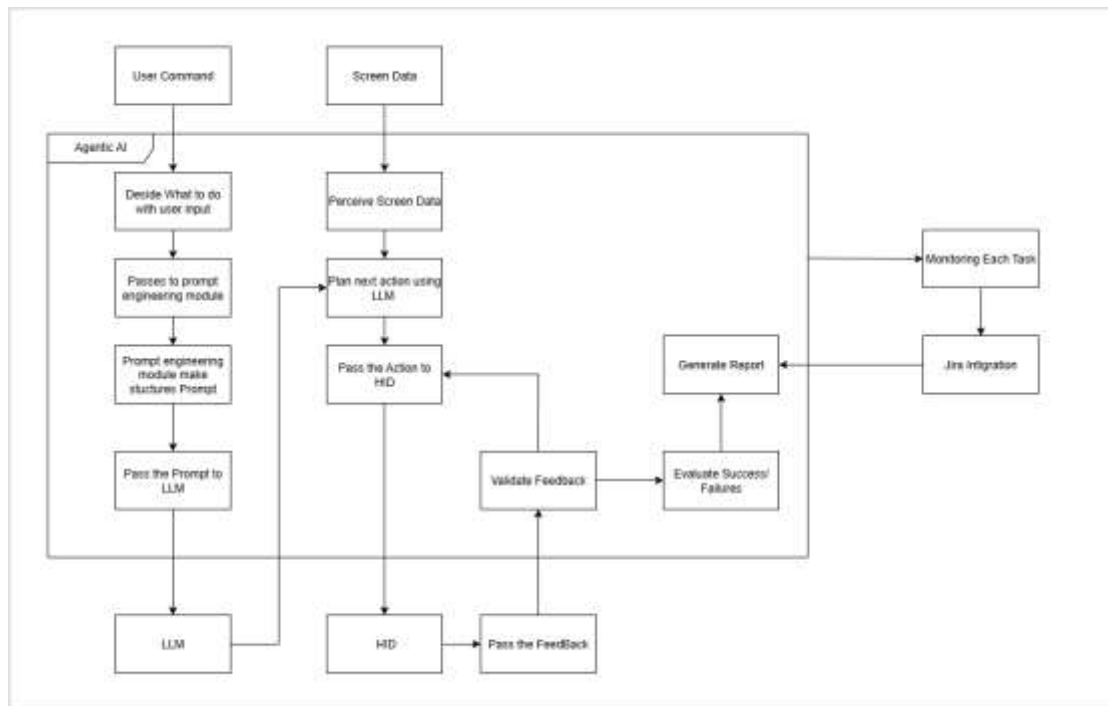
10.3 Appendix C: Flow Chart Diagram



10.4 Appendix D: Block Diagram



10.5 Appendix E: Agentic AI Logic Block diagram



10.6 Appendix F: Tools and Technologies

- LLM Platforms: OpenAI GPT-4, LLaMA
- Vision Modules: OpenCV, Tesseract OCR
- Automation Frameworks: Robot Framework, SikuliX
- Development Environment: Python, Docker